

Bus tracker — hardware integration spec

For the ESP32 + GPS + cellular tracker units fitted to the six campus buses.

Plain HTTP. No TLS, no certificates. A relay accepts your unencrypted request and forwards it to the database over HTTPS. Because the link to you is unencrypted, each unit signs its data with a secret that is never transmitted.

Everything below was tested against the live relay; the responses shown are real ones.

1. What the device has to do

In one sentence: **get a GPS fix, sign it, POST it every 5 seconds.**

```
loop:
  read GPS
  build JSON body (with an ever-increasing counter)
  sig = HMAC_SHA256(device_secret, body)
  POST it
  wait 5 s
```

That is the whole job. No route logic, no map data, no stop coordinates, no claiming, no TLS. The server works out which route the bus is on, whether it is parked, and which way it is going.

2. Endpoint

URL	http://recverse.ibinujaleel.dev:8081/p
Method	POST
Port	8081, plain HTTP
Content-Type	application/json

Health check (no signature needed, useful for bring-up):

GET http://recverse.ibinujaleel.dev:8081/health → {"ok":true,...}

3. The request

```
POST /p HTTP/1.1
Host: recverse.ibinujaleel.dev:8081
X-Sig: 9f2b7c...64 hex chars
Content-Type: application/json
Content-Length: 84

{"b":"bus1","d":"esp32-01","c":1234,"lat":11.3185,"lng":75.9379,"spd":6.4,"hdg":312}
```

Field	Type	Meaning
b	string	Bus id. Fixed per unit — see §4.

Field	Type	Meaning
d	string	Device id. Fixed per unit — see §4.
c	integer	Counter. Must increase on every request, forever. See §5.
lat / lng	number	Decimal degrees, WGS84. 6 decimals is plenty.
spd	number	Metres per second. NMEA gives knots — multiply by 0.514444.
hdg	number	Degrees 0–360, 0 = north. Send <code>null</code> if unknown — never 0.

Success:

```
{"ok": true}
```

Rejections — check the HTTP status:

Status	Body	Meaning
200	<code>{"ok":true}</code>	Accepted
403	bad signature	HMAC did not match — see §5
403	wrong bus	This device is not bound to that bus id
403	unknown device	d is not in the relay's device list
409	replay	Counter was not higher than the last one
400	bad json / bad coords	Malformed request
502	upstream down	Relay is up, database is not — retry later

Do not send a timestamp. The server stamps arrival time itself, so a wrong RTC cannot make a live bus look offline.

4. Per-unit identity

Each unit is flashed with its own id, bus and secret. ****A unit gets only its own secret.**** These are issued separately — they are not in this document.

Device id	Bus	Secret
esp32-01	bus1	issued separately
esp32-02	bus2	issued separately
esp32-03	bus3	issued separately
esp32-04	bus4	issued separately
esp32-05	ac_mbh	issued separately
esp32-06	ac_lh	issued separately

The secret is a 32-character hex string. Store it in NVS, not in a source file that ends up on GitHub.

5. Signing — the important part

```
X-Sig = hex( HMAC_SHA256( device_secret, exact_body_bytes ) )
```

Sign the **exact bytes you transmit**. Build the JSON string once, sign that string, send that string. Re-serialising between signing and sending is the usual cause of a `bad signature` — a different key order or an extra space changes the hash completely.

The secret is never sent. Someone sniffing the network can read bus positions but cannot forge one.

The counter

`c` must be strictly greater than the previous accepted value, forever, per device. It stops someone from recording a valid request and replaying it later.

Store it in NVS and increment on every send. **Persist across reboot** — if it resets to 0 the relay will reject everything with `409 replay` until it climbs past the old value. A simple, safe scheme:

```
c = unix_seconds_from_GPS // monotonic, survives reboot for free
```

If you have no reliable clock, use an NVS counter and bump it by 100 on boot to cover any unwritten increments.

ESP32 notes

`MBEDTLS_MD_HMAC()` with `MBEDTLS_MD_SHA256`, or Arduino's `MBEDTLS` wrapper.

The ESP32 has SHA-256 in hardware; signing takes microseconds and adds 64 bytes plus the header name. Do not roll your own HMAC.

6. Constraints

6.1 Interval: 5 seconds

Not arbitrary. Stop-arrival detection needs at least two fixes inside a 40 m radius; at ~25 km/h a bus crosses that in ~11 s. At 10 s intervals you get one fix and the app starts missing arrivals. ****Do not go slower than 5 s without telling us.****

6.2 Reuse the TCP connection

The relay speaks HTTP/1.1 keep-alive. Open one socket and keep it. Reconnecting every 5 s multiplies data use and power draw for no benefit.

6.3 Data budget

~350 bytes per round trip over a 10.9-hour service day:

Interval	Per bus/day	Per bus/month	All 6/month
5 s	2.8 MB	83 MB	0.5 GB
10 s	1.4 MB	41 MB	0.25 GB

A 500 MB/month SIM is comfortable. (Dropping TLS cut this by roughly 60% compared with talking to the database directly.)

6.4 Buffer when offline

Campus has dead spots. Hold unsent fixes in a RAM ring buffer (~200 entries) and flush on reconnect. Keep the counter increasing across buffered sends.

Talk to us before implementing backfill — the API needs a small change to accept timestamped historical points.

Never block the GPS loop on a network call.

6.5 Power

Buses cut power with the ignition; expect 12–24 V, dirty, with load-dump spikes. Use an automotive-rated buck converter, and a supercapacitor or small LiPo so the unit can finish its current request and shut down cleanly.

6.6 Module choice

Since there is no TLS, almost any module works — **SIM800L is fine here**, as are SIM7600 / A7670 / SIM7670. Requirements: TCP client, ~1 KB buffers, HTTP/1.1 keep-alive.

7. Bring-up checklist

On the bench, before fitting anything to a bus:

1. `GET /health` → `{"ok":true,...}`. Proves routing and the SIM data plan.
2. One signed `POST /p` → `{"ok":true}`.
3. Send the same body twice → second must give `409 replay`. Proves the counter works.
4. Corrupt one byte of the signature → `403 bad signature`.
5. Ask us to confirm the bus appears on the tracker map within ~10 s.
6. Power-cycle → resumes with no manual step, no `409` storm.
7. Pull the antenna → keeps retrying; no lock-up, no reboot loop.
8. Run 30 minutes → stable memory, connection still alive.

Reference request (replace the secret, then compare against your firmware's bytes):

```
BODY='{ "b": "bus1", "d": "esp32-01", "c": 1, "lat": 11.3185, "lng": 75.9379, "spd": 6.4, "hdg": 312 }'  
SIG=$(printf '%s' "$BODY" | openssl dgst -sha256 -hmac "YOUR_SECRET_HERE" -hex | awk '{print $2}')  
curl -X POST http://recverse.ibinujaleel.dev:8081/p \\  
-H "X-Sig: $SIG" -H "Content-Type: application/json" -d "$BODY"
```

8. Please do NOT

- Send speed in km/h or knots. **Metres per second.**
- Send `hdg: 0` when heading is unknown — send `null`. Zero means north and will point every parked bus the same wrong way on the map.
- Reset the counter on reboot. Everything will be rejected until it catches up.
- Re-serialise the JSON after signing it. Sign and send the same bytes.
- Share a secret or a device id between units.
- Invent bus ids. Only the six in §4 exist.
- Commit secrets to a public repo.

Questions come to us rather than being guessed at — a wrong assumption here costs a site visit to six buses.